Churn Preventer Agent: Architecture & Build Plan

Sep 23, 2026 · @Someone

1. Thesis and MVP scope

We build a checkout-focused pipeline that finds friction and silent data bugs in live rrweb sessions, then ranks them by revenue at risk. Frustration tools already flag rage clicks. Our wedge is what they miss: business-rule violations (a cart total that does not match its line items) and retroactive detection (ship a new rule, re-scan the past 30 days).

One working target for the demo: a bug hit in the browser appears as a ranked finding on the dashboard in under 60 seconds. This is our design target, not a measured figure.

In scope for the MVP

- A browser SDK that wraps rrweb, masks sensitive fields, and ships events in compressed batches.
- A GCP ingest and processing pipeline (Cloud Run, Pub/Sub, Cloud Storage, BigQuery).
- Four pluggable detectors: rage click, dead click, invariant check (cart total vs line items), and JS error or failed-request correlation.
- Cross-session clustering, then Gemini triage and root-cause hypotheses on clusters only.
- A bitemporal findings store with a retroactive re-scan when a rule changes.
- A dashboard with a findings feed, embedded replay, and revenue-at-risk estimates.

Deliberately out of scope

- Heatmaps, funnels, and general product analytics.
- Mobile SDKs and multi-tenant billing.
- Auto-fixing customer code.
- Self-serve onboarding beyond pasting one script tag.

We target the Retail & Commerce theme and keep every feature tied to the checkout flow, so the story stays narrow and demonstrable.

2. System architecture

Raw rrweb data is stored cheaply and never queried directly; a tiered pipeline turns it into signals, clusters, and ranked findings, and only the last tier touches an LLM.

```
flowchart LR
  A["Browser SDK<br/>rrweb + masking"] -->|"gzip batches"|
B["Collector<br/>Cloud Run"]
  B --> C[("Raw chunks<br/>Cloud Storage")]
  B --> D["Pub/Sub<br/>events.raw"]
  D --> E["Processor<br/>detectors, Cloud Run"]
  E <--> F[("Hot session state<br/>Firestore")]
  E --> G[("BigQuery<br/>signals, bitemporal")]
  E -->|"signals topic"| H["Clusterer<br/>fingerprints"]
  H --> I["Agent service<br/>Gemini via ADK"]
  I --> J[("Findings<br/>BigQuery + Firestore")]
  J --> K["Dashboard + API<br/>Cloud Run / Firebase"]
  C -.->|"replay"| K
```

Read left to right: the SDK ships batches, the collector persists them and fans out, the processor runs deterministic detectors, and the agent enriches only clustered candidates before they reach the dashboard.

Component	Responsibility	Scales by
Browser SDK	Wrap rrweb, mask inputs, detect cheap local triggers, batch and compress, flush with sendBeacon on unload	Client-side sampling
Collector (Cloud Run)	Validate site key, assign chunk sequence, write raw chunk to Cloud Storage, publish a small envelope to Pub/Sub	Concurrent requests per instance
Pub/Sub (events.raw)	Decouple ingest from processing; ordering key = session_id	Partitioned throughput, backlog buffer
Processor (Cloud Run)	Rebuild DOM state per chunk, run every registered detector, emit signals; keep a short session tail in Firestore	Instances subscribed to the topic

Component	Responsibility	Scales by
BigQuery	Append-only signals and findings with event time, ingest time, and rule version	Partition by event date, cluster by tenant and session
Clusterer	Group signals across sessions by fingerprint (page, element selector, error signature, rule id)	Micro-batches fed by the signals topic
Agent service	Gemini triage, root-cause hypothesis, ranking against the rule set, rule authoring	Runs per cluster, not per session
Dashboard + API	Findings feed, replay player reading raw chunks, revenue at risk	Static hosting plus a thin API

Design rules

- Every stage is idempotent, keyed by session_id and chunk sequence, so Pub/Sub redelivery is harmless.
- Raw data is immutable. Anything derived can be recomputed, which is what makes retroactive re-scan possible.
- The LLM sees structured evidence (signals, selectors, DOM diffs, a few screenshots), never raw rrweb streams.

3. Data model

Five append-only tables carry everything: a raw-chunk index, signals, business events, rules, and findings, with two clocks on every row so past states can be reconstructed.

Ingest envelope (one SDK batch; the rrweb events stay opaque to the collector):

```
{
  "site_key": "pk_live_...",
  "session_id": "s_8f2c...",
  "seq": 42,
  "client_ts_start": 1790000000000,
  "client_ts_end": 1790000004800,
  "client_sent_at": 1790000004900,
  "sdk_version": "0.3.1",
  "release": "git:a1b2c3d",
  "page": { "route": "/checkout", "url_path": "/checkout" },
  "events": [ /* rrweb events: FullSnapshot, IncrementalSnapshot, Meta,
Custom */ ],
  "business_events": [ { "name": "cart_viewed", "value_minor": 129900,
"currency": "INR" } ]
}
```

The `release` field lets us tie a finding to a deploy. Business events are explicit custom events the site emits (cart viewed, checkout started, order placed); they feed revenue-at-risk.

Table	Grain	Key columns
raw_chunks	One row per stored batch	tenant_id, session_id, seq, gcs_uri, bytes, event_time_start, event_time_end, ingest_time
signals	One row per detector hit	signal_id, tenant_id, session_id, detector, detector_version, rule_version, fingerprint, route, element_selector, evidence (JSON), event_time, ingest_time
business_events	One row per commerce event	tenant_id, session_id, name, value_minor, currency, event_time, ingest_time
rules	One row per rule version	rule_id, version, kind (builtin or generated), definition (JSON), created_by, recorded_at
findings	One row per finding version	finding_id, fingerprint, severity, revenue_at_risk_minor, affected_sessions, summary, evidence_refs, rule_version, model_version, valid_from, valid_to, recorded_from, recorded_to

The two clocks

- Valid time (`event_time`, `valid_from`, `valid_to`): when it happened in the user's browser, corrected for clock skew by comparing `client_sent_at` with the collector's receive time.
- Transaction time (`ingest_time`, `recorded_from`, `recorded_to`): when our system learned it or wrote that version.

Findings are never updated in place. A change writes a new row and sets `recorded_to` on the old one, so "what did we believe last Tuesday" is a filter, not a backup restore.

4. Detection engine

Detectors are small plugins that read a normalized event stream, not raw rrweb, so rage click detection is one module among several and adding a fifth detector touches no other code.

The processor first converts rrweb events into a normalized stream: clicks (time, target selector, position), DOM mutation counts per target, navigations, inputs, plus custom events from the SDK for network calls, JS errors, and probe values. Every detector implements one interface:

```
interface Detector {
  id: string;              // "rage_click"
  version: string;         // bumped on any logic change
  params: Record<string, number | string>;
  run(ctx: SessionContext, chunk: NormalizedChunk): Signal[];
}

interface Signal {
  detector: string;
  fingerprint: string;     // hash(detector, route template, normalized
selector, error signature)
  eventTime: number;       // valid time, ms
  evidence: {              // small, structured, replayable
    sessionId: string; seq: number; tStart: number; tEnd: number;
    selector?: string; values?: Record<string, unknown>;
  };
}
```

Detector	Fires when (starting defaults, to be tuned)	Covers
rage_click	3 or more clicks on the same element within 1.5 s and 40 px	Users hammering a broken control
dead_click	Click on an interactive element with no DOM mutation, navigation, or request within 800 ms	Controls that do nothing
response_decay	An element responded to its first k clicks, then 3 or more consecutive clicks get no response	The button that stops registering after 5 clicks
invariant_check	A rule over probe values is violated beyond tolerance (for example cart total differs from the sum of line items by more than 1 minor unit)	Silent data bugs with no user frustration signal
error_correlation	A JS error, unhandled rejection, or 4xx/5xx/timeout occurs within 2 s before another signal	Attaches root-cause evidence and raises severity

These thresholds are our own starting values, not industry standards; expose them as `params` per tenant.

Probes make invariants cheap and re-scannable. A tenant registers probe selectors once (cart total, line-item price, quantity, discount). The SDK emits their text values as custom events whenever the DOM under them changes. Invariants are then pure functions over probe values: no DOM rebuild, and any new rule over already-captured probes can be re-scanned from BigQuery alone. A rule that needs a selector nobody probed goes to the slower rebuild path (Section 6).

Fingerprints are what let 400 sessions collapse into one finding. Normalize selectors before hashing: strip generated ids, hashed class names, and positional indexes, and replace dynamic route segments with templates.

5. Agent layer (Gemini + ADK)

Deterministic code decides what is a finding and how much it is worth; Gemini explains it, verifies it visually, and writes new rules. That split keeps rankings reproducible and puts the Gen AI where it adds the most.

Build it with the Agent Development Kit, which now ships a TypeScript 2.0 release with graph workflows that mix deterministic steps and LLM steps, so the whole stack can stay in one language. Deploying is `npx adk deploy cloud_run` with the agent exported as `rootAgent` in `agent.ts`. As of 23 Sep 2026 the GA Gemini models include `gemini-3.8-flash` and the cheaper `gemini-3.5-flash-lite`; model IDs turn over roughly monthly, so read them from config.

Agent	Model tier	Input	Output
Triage	flash-lite	Cluster summary: fingerprint, count, route, top selectors, release, 3 to 5 sample evidence items	JSON: title, category, confidence, likely cause, repro steps
Root cause and verify	flash (reasoning, vision)	Triage output plus tool calls: session samples, DOM diff at time t, error context, screenshot at time t, release diff	JSON: cause hypothesis, visual confirmation yes or no, evidence ids cited
Rule author	flash	A merchant sentence such as "cart total must equal line items minus discount plus shipping"	Rule JSON (probes and invariant), dry-run precision on recent sessions, then human approval
Ranker	Deterministic core with a bounded LLM step	Enriched finding and the priority rule set	Score, tier, and the priority rule ids that applied

Ranking against a rule set. The priority rule set is versioned data in the `rules` table, for example: checkout and payment blockers outrank pricing errors, pricing errors outrank browse-page friction, and a regression appearing right after a release gets a multiplier.

Gemini classifies each finding into these buckets and cites the rule ids. The score itself is computed in code:

```
\text{RaR} = N_{\text{affected}} \times \bar{V}_{\text{cart}} \times
\left(a_{\text{affected}} - a_{\text{baseline}}\right)
```

Here N is affected sessions that had a cart, V is their average cart value, and a is the checkout abandonment rate for affected sessions versus a baseline of unaffected sessions on the same route. Using the difference, not the raw abandonment, avoids claiming revenue that would have been lost anyway. The LLM may move a finding by at most one severity tier, and only with a cited rule id.

Grounding and guardrails

- Every claim in an LLM output must cite evidence ids we supplied; a validator rejects outputs that cite unknown ids.
- Outputs use structured JSON schemas, never free text parsed with regex.
- The visual check is the false-positive filter: render the frame from the replay at the signal time and ask Gemini whether the on-screen state actually shows the defect.
- Each finding stores prompt hash, model id, and rule version so any verdict can be reproduced later.

6. Bitemporal search and retroactive re-scan

Retroactive re-scan is the feature to demo: change or add a rule, re-run it over the past 30 days, and see which users were already hurt, without overwriting what the system believed at the time.

Three kinds of question, each answered by filtering one or both clocks from Section 3:

Question	Filters on	Example
When did users hit this?	Valid time	Sessions with the coupon bug between 1 and 14 Sep
What did we know then?	Transaction time	What the dashboard showed on 15 Sep
What did we know then about then?	Both	On 15 Sep, what did we think happened 1 to 14 Sep, versus what we know now

The third row is the one I did not find any competitor marketing: the gap between the two answers is the bugs the old rules missed.

```
-- Findings as we believed them on 15 Sep about events during 1-14 Sep
SELECT finding_id, severity, revenue_at_risk_minor
FROM findings
WHERE valid_from < TIMESTAMP('2026-09-14')
  AND (valid_to IS NULL OR valid_to > TIMESTAMP('2026-09-01'))
  AND recorded_from <= TIMESTAMP('2026-09-15')
  AND (recorded_to IS NULL OR recorded_to > TIMESTAMP('2026-09-15'));
```

Re-scan flow

1. A rule is created or edited, producing a new row and version in `rules`. A dry run over the last 1,000 sessions shows hit rate and sample hits.
2. A human approves; a Cloud Run Job scans the chosen window, 30 days by default.
3. New signals are written with the new `rule_version`, their original `event_time`, and `ingest_time` = now.
4. The clusterer writes new finding versions and closes the old ones by setting `recorded_to`.
5. The dashboard tags each finding as known at the time or found later, with sessions affected since the bug began.

Two execution paths

- Fast path: rules over signals, probe values, or the normalized event stream. The processor also writes a compact Parquet copy of the normalized stream to Cloud Storage as a derived cache, rebuildable from raw, exposed to BigQuery as an external table. Re-runs take minutes.
- Rebuild path: a rule needing a selector nobody probed. A Cloud Run Job replays raw chunks in headless Chromium using the rrweb replayer and evaluates the new selector. It is slower and costs more, so cap it per run and treat it as phase 2 unless time allows.

Cost guard: partition pruning by event date and a maximum-bytes-billed setting on every re-scan query.

7. Dashboard and API

The dashboard is a ranked findings feed with one click from any finding to the exact replay moment, plus a time-travel control that makes the bitemporal model visible.

Views

- Overview: revenue at risk over the last 7 days, open findings by tier, sessions ingested, and measured detection latency.
- Findings feed: ranked cards showing title, tier, revenue at risk, affected sessions, first and last seen, release, and a known-at-the-time or found-later tag.
- Finding detail: Gemini summary and repro steps, the visual-verification screenshot, related errors, and evidence items that open the rrweb player at the signal time.
- Rules: detector parameters, the probe registry, and plain-English rule authoring with a dry-run result before activation.
- Time travel: an as-of date and a valid-time range that re-query the feed, so a viewer can watch the retroactive re-scan change the picture.

API surface (REST on Cloud Run)

Endpoint	Purpose
POST /v1/ingest	Collector; site-key auth, gzip batches
GET /v1/findings	List with filters: status, tier, route, release, as_of, valid_from, valid_to
GET /v1/findings/{id}	One finding with its version history
GET /v1/findings/{id}/evidence	Signals and short-lived signed replay URLs
POST /v1/rules	Create a rule version, including plain-English drafts
POST /v1/rules/{id}/dry-run	Hit rate and samples over recent sessions
POST /v1/rules/{id}/rescan	Start a retroactive re-scan job for a window

Read path. BigQuery is the source of truth, but it is a poor fit for low-latency UI reads. The agent service also writes the latest version of each finding to Firestore, and the feed subscribes to it for live updates. Time-travel queries go to BigQuery.

Suggested front end: React and Vite on Firebase Hosting, calling the Cloud Run API.

8. Privacy and security

Treat privacy as an architecture constraint: nothing sensitive should leave the browser, and nothing sensitive should reach Gemini. rrweb's defaults are permissive, so masking has to be configured deliberately.

According to the rrweb guide, `maskAllInputs` defaults to false and only password inputs are masked by default. A GitHub issue against rrweb 2.0.0-alpha.18 also reports hidden inputs escaping `maskAllInputs`, open when checked. Verify against the version you pin, and do not rely on client masking alone.

Layer	Control	Enforced in
Capture	`maskAllInputs: true`; `blockSelector` for payment and identity elements; `maskTextSelector` for account, address, and order-confirmation text; `maskInputFn` and `maskTextFn` for pattern masking of emails, phone numbers, and card-like digit runs	Browser SDK
Probes	Allowlisted selectors only, values parsed to numbers and currency before leaving the browser; probe text is never sent as text	Browser SDK
URLs	Drop query strings and fragments from `url_path` by default	Browser SDK
Ingest scrub	Second-pass scrubber over the batch JSON before it is written to Cloud Storage; reject unknown site keys; fail closed on parse errors	Collector
LLM boundary	Gemini receives structured evidence and screenshots rendered from the already-masked replay, never raw HTML	Agent service
Tenancy	`tenant_id` on every row; per-tenant Cloud Storage prefix; BigQuery row-level policies or per-tenant authorized views; replay URLs are short-lived signed URLs	Data and API

Layer	Control	Enforced in
Retention	Lifecycle rule on raw chunks (30 days as a starting default, configurable per tenant); signals and findings carry no page content and can live longer; delete by session_id through the raw_chunks index	Storage

Region. Keep the Cloud Storage bucket, BigQuery dataset, and Cloud Run services in one region near the tenants, for example asia-south1 or asia-southeast1, so data residency is a config choice rather than a rewrite.

Consent. The SDK should start only after the host site's consent banner allows it. Rules differ by country across JAPAC, so get legal review before any real customer data flows; for the hackathon, run on synthetic sessions from the demo shop only.

9. Scaling and cost controls

The architecture scales because volume shrinks at every tier: many sessions become fewer signals, signals become clusters, and only clusters reach the LLM. The order in which things break, and the fix for each, is below.

Stage	What breaks first	Mitigation
Browser SDK	Payload size and main-thread cost on heavy pages	Use rrweb `sampling` to throttle mouse-move and scroll, `packFn` compression, periodic checkouts, and a hard cap on batch bytes
Collector	Bursts at campaign or sale time	Cloud Run autoscaling with a minimum instance count; body size limit; idempotency on session_id and seq
Pub/Sub	Large messages and head-of-line blocking on a hot session	Messages carry the Cloud Storage URI and metadata only, never the events; ordering key is session_id
Processor state	Firestore read and write per chunk becomes the bottleneck and the bill	Cache the session tail in memory with a TTL, write only on change, move to Memorystore if needed

Stage	What breaks first	Mitigation
BigQuery	Many small inserts and full-table scans	Batch through the Storage Write API; partition by event date; cluster by tenant_id and session_id; never query raw chunks from BigQuery
Clusterer	Fingerprint cardinality explosion	Selector normalization, a per-tenant cluster cap, and only the top 5 evidence items stored per cluster
Agent and LLM	Cost and rate limits	Run per cluster, skip re-triage when a cluster is unchanged for the same release, add backoff, and process top-N first
Dashboard	BigQuery latency on live views	Firestore hot copy of the latest finding versions

Edge sampling with a ring buffer. The SDK keeps the last 30 seconds of events in memory and ships a session only when a local trigger fires (rage or dead click, JS error, failing probe), plus an unbiased baseline sample of ordinary sessions. Do not skip the baseline: revenue at risk in Section 5 needs unaffected sessions to compare against, and trigger-only sampling would bias abandonment upward. Record the sampling rate on each session so counts can be re-weighted.

Why the funnel matters for cost. Gemini 3.8 Flash is listed at $0.75 per million input tokens and $3.75 per million output tokens through 31 Dec 2026, rising to $1.50 and $7.50 on 1 Jan 2027. With illustrative assumptions of 6,000 input and 1,000 output tokens per call, triaging 200 clusters a day costs about $1.65. Sending each of 1,000,000 daily sessions through the same call would cost about $4,500 in input tokens alone. Treat both as lower bounds, since the model can spend more tokens on hard tasks.

Beyond one tenant. Give large tenants their own Pub/Sub subscription and Cloud Run concurrency limits, enforce a per-site-key quota, and keep every table keyed by tenant_id so a noisy customer cannot starve the rest. Cloud Logging is the wrong sink for this data: a log entry is capped at 256 KiB and the limit cannot be raised; use it only for our own service logs.

10. GCP service map and repository layout

Everything runs on Cloud Run plus managed data services, which satisfies the Cup requirement to use Gemini or Google's agent platforms and to deploy on Cloud Run or Firebase.

Need	GCP service	Notes
Collector, processor, clusterer, agent, API	Cloud Run	One service each; the agent deploys with `npx adk deploy cloud_run`
Event fan-out	Pub/Sub	Topics: events.raw and signals; ordering key = session_id
Raw chunks and Parquet cache	Cloud Storage	Per-tenant prefix, lifecycle rule for retention
Signals, findings, rules, business events	BigQuery	Partitioned by event date, clustered by tenant_id and session_id
Hot session tail and latest findings	Firestore	Session tail with a TTL; live dashboard feed
Re-scan and rebuild work	Cloud Run Jobs	Started by the API; capped per run
Dashboard hosting	Firebase Hosting	React and Vite build
API keys and site keys	Secret Manager	Gemini key held here per the ADK Cloud Run docs
Our own service logs and alerts	Cloud Logging and Monitoring	Never customer session data

Gemini can be reached with an AI Studio API key for speed during the hackathon, or through Google's agent platform with service-account auth for a production posture. Both are named as acceptable in the Cup rules; I have not compared their quotas.

Monorepo layout (TypeScript throughout; detectors are shared by the SDK and the processor)

```
churn-preventer/
  apps/
    sdk/            browser SDK: rrweb wrapper, probes, ring buffer, local
triggers
    collector/      Cloud Run: validate, scrub, write to GCS, publish to
Pub/Sub
    processor/      Cloud Run: normalize, run detectors, emit signals
    clusterer/      Cloud Run: fingerprint grouping, scoring, Firestore
updates
    agent/          ADK agents: triage, root cause, rule author, ranker
    api/            Cloud Run REST API
    dashboard/      React + Vite on Firebase Hosting
    demo-shop/      e-commerce demo with switchable bug flags
  packages/
    schema/         shared types: envelope, Signal, Finding, Rule
    normalizer/     rrweb events to normalized stream
    detectors/      rage_click, dead_click, response_decay, invariant_check,
error_correlation
  sim/              Playwright scripts that generate synthetic sessions, with
and without bugs
  infra/            scripts for topics, buckets, datasets, IAM
```

The `sim` folder matters as much as any service: it produces the hundreds of realistic sessions that make the dashboard and the retroactive re-scan demonstrable.

11. Build plan to submission

The Cup's home page lists the building phase as 7 Sep to 18 Oct 2026, with submission due 18 Oct, so we have 25 days from today; build a thin vertical slice first and add depth in layers. Its FAQ says registration closes 4 Oct while the home page says team formation runs to 11 Oct, so register by 4 Oct to be safe. The home page also lists eligibility as ages 21 and over, working professionals, entrepreneurs, and startups, with students ineligible: confirm this applies to your whole team before investing more time.

Dates	Phase	Deliverable	Exit test
23 to 25 Sep	0. Foundations	Team registered, GCP project, monorepo, shared schema package, infra scripts for topics, bucket, and datasets, demo shop skeleton with bug flags	`infra` creates everything from scratch in one command
26 Sep to 1 Oct	1. Vertical slice	SDK to collector to Pub/Sub to processor with one detector (dead_click) to BigQuery to a bare findings list	Clicking the broken button in the demo shop produces a signal row within a minute
2 to 7 Oct	2. Detector set	rage_click, response_decay, error_correlation, probes and invariant_check; clusterer with fingerprints; Firestore live feed; `sim` generating a few hundred sessions	The three injected bugs each collapse into one cluster
8 to 12 Oct	3. Agent layer	Triage, visual verification, revenue-at-risk score, priority rule set; finding detail page with replay at the signal moment	A finding shows a Gemini summary, repro steps, and a working replay link
13 to 15 Oct	4. Bitemporal	Versioned findings, time-travel controls, plain-English rule author with dry run, fast-path retroactive re-scan	Adding a rule surfaces past sessions tagged found-later
16 to 17 Oct	5. Harden and package	Final deploy to Cloud Run and Firebase, 3-minute video, deck, README, public repo cleanup	A fresh browser runs the whole demo with no console errors
18 Oct	Submit	Prototype link, video, public repo, deck	Submitted a day early if possible

Demo shop with switchable bug flags

- `BUG_DEAD_BUTTON`: the Apply Coupon button silently stops responding after the fifth click.
- `BUG_CART_TOTAL`: the cart total ignores quantity on one line when a discount is applied, so it differs from the sum of line items.
- `BUG_CHECKOUT_ERROR`: the payment step throws a JS error and a 500 for one region, producing rage clicks and abandonment.

The `sim` generator should run several hundred scripted sessions with randomized behavior, where bug-affected sessions abandon at a higher rate than the rest. Those rates are parameters we choose, so say clearly in the demo and README that the data is synthetic.

Splitting the work (fold tracks together if the team is two people)

- Track A, capture and ingest: SDK, probes, ring buffer, collector, infra.
- Track B, detection: normalizer, detectors, processor, clusterer.
- Track C, intelligence: ADK agents, scoring, rule author, bitemporal queries and re-scan.
- Track D, product and demo: dashboard, demo shop, `sim`, video and deck.

12. Demo script and risks

The 3-minute video should spend about a third of its time on the retroactive re-scan, because that is where innovation and technical merit meet. The Cup scores technical merit and Gen AI implementation at 40%, problem alignment and impact at 25%, innovation and creativity at 25%, and user experience at 10%.

Time	Show	Say	Scores on
0:00 to 0:20	Demo shop cart where the total does not match its lines	Users do not report this; they leave	Problem alignment
0:20 to 0:50	Click Apply Coupon six times; a quick architecture frame	One script tag streams rrweb sessions through Pub/Sub into our detectors	Technical merit

Time	Show	Say	Scores on
0:50 to 1:30	Findings feed: three findings from hundreds of sessions; open one	Clustered, ranked by revenue at risk; Gemini explains it and confirms it visually; jump to the replay moment	Gen AI implementation
1:30 to 2:10	Type a plain-English rule; dry run; approve; re-scan	A new rule finds last week's sessions nobody knew were broken	Innovation
2:10 to 2:40	Move the as-of control	What we believed then versus what we know now	Innovation and UX
2:40 to 2:55	Funnel counts: sessions, signals, clusters, Gemini calls	The LLM only sees clusters, so cost stays flat as traffic grows	Technical merit and impact

Record from pre-warmed services and keep a pre-recorded fallback clip.

Top risks

Risk	Mitigation or fallback
Judges say existing session-replay tools already do this	Concede that rage-click detection is common; lead with business-rule invariants and retroactive re-scan, and name the gap plainly
The rebuild path (headless replay) consumes the schedule	Keep the demo on the fast path; treat rebuild as phase 2
False positives from detectors	Visual verification, tunable parameters, dry run, and precision measured against sim ground truth
Revenue at risk looks made up on synthetic data	State it is synthetic, show the formula, and use the affected-versus-baseline difference
Live demo failure from cold starts, quota, or model latency	Minimum instances, cache Gemini output per fingerprint and release, fallback recording

Risk	Mitigation or fallback
Gemini model IDs change or move to preview	Read model IDs from config and pin to GA models
Eligibility or registration dates differ from what we assumed	Confirm on the official pages this week and register by 4 Oct

Sources

Pages opened while preparing this plan, as of 23 Sep 2026.

- AI Builder Cup 2026: home, timeline and eligibility
- AI Builder Cup 2026: FAQs
- AI Builder Cup 2026: themes and judging criteria
- Gemini API: latest model
- Gemini API: release notes
- Agent Development Kit overview
- ADK: deploy to Cloud Run
- rrweb guide (record options)
- rrweb issue 1609: hidden inputs not masked
- rrweb: best open source session replay tools
- Cloud Logging quotas and limits